Full-text search+extensions

2026. February 27., Friday 15:49

The LIKE operator

_ wildcard: Used to match exactly one character.

% wildcard: Used to match zero or more characters.

```
SELECT title
FROM film
WHERE title LIKE 'ELF%';
```

```
+----------------------+
| title                |
+----------------------+
| ELF PARTY            |
+----------------------+
```

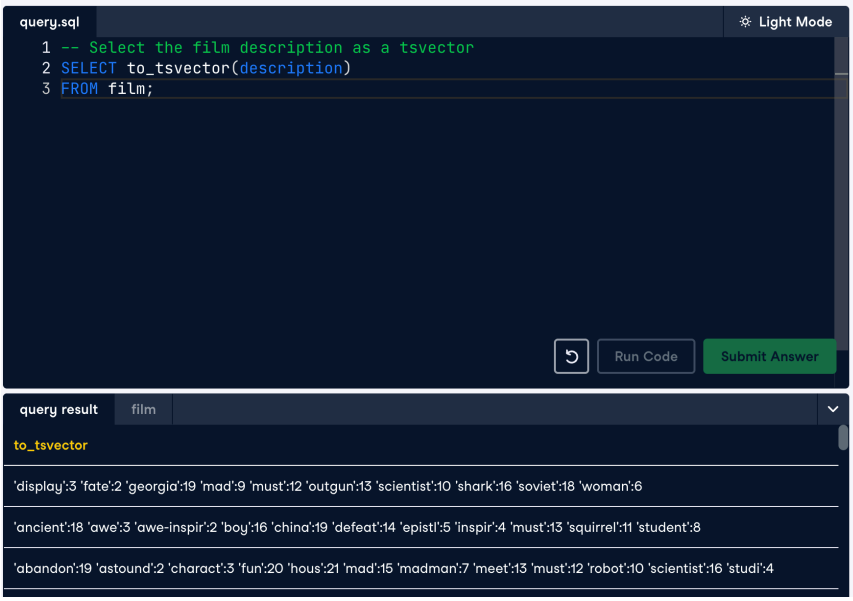
LIKE is case sensitive

LIKE versus full-text search

```
SELECT title, description
FROM film
WHERE to_tsvector(title) @@ to_tsquery('elf');
```

```
+----------------------+
| title                |
+----------------------+
| ELF PARTY            |
| ENCINO ELF           |
| GHOSTBUSTERS ELF     |
+----------------------+
```

@@: match operator, compares
This is not case sensitive
To_tsvector and to_tsquery convert to a tsvector data type, which is a sorted list of words that have been normalized into variants of the same word - these are called lexemes



User-defined data types

Enumerated data types

```
CREATE TYPE dayofweek AS ENUM (
```

```
CREATE TYPE dayofweek AS ENUM (
      'Monday',
      'Tuesday',
      'Wednesday',
      'Thursday',
      'Friday',
      'Saturday',
      'Sunday'
);
```

These enumerated types are custom that are never going to change

Getting information about user-defined data types

```
SELECT typname, typcategory
FROM pg_type
WHERE typname='dayofweek';
```

```
+-----------+-------------+
| typname   | typcategory |
|-----------|-------------|
| dayofweek | E           |
+-----------+-------------+
```

E represents enum type

Getting information about user-defined data types

```
SELECT column_name, data_type, udt_name
FROM INFORMATION_SCHEMA.COLUMNS
WHERE table_name ='film';
```

```
+-----------------------------------------------+
| column_name | data_type         | udt_name    |
|-------------|-------------------|-------------|
| title       | character varying | varchar     |
| rating      | USER-DEFINED      | mpaa_rating |
+-----------------------------------------------+
```

User-defined functions

```
CREATE FUNCTION squared(i integer) RETURNS integer AS $$
    BEGIN
        RETURN i * i;
    END;
$$ LANGUAGE plpgsql;
```

```
SELECT squared(10);
```

```
+---------+
| squared |
|---------|
| 100     |
+---------+
```

EXTENSIONS

Intro to PostgreSQL extensions

Commonly used extensions

- PostGIS
- PostPic
- fuzzystrmatch
- pg_trgm

Querying extension meta data

Available Extensions

```
SELECT name
FROM pg_available_extensions;
```

```
+--------------------+
| name               |
|--------------------|
| dblink             |
| pg_stat_statements |
+--------------------+
```

Installed Extensions

```
SELECT extname
FROM pg_extension;
```

```
+----------+
| name     |
|----------|
| plpgsql  |
+----------+
```

```
--Enable the fuzzystrmatch extension
CREATE EXTENSION IF NOT EXISTS fuzzystrmatch;
--Confirm that fuzzstrmatch has been enabled
SELECT extname FROM pg_extension;
```

```
+---------------+
| name          |
|---------------|
| plpgsql       |
| fuzzystrmatch |
+---------------+
```

Fuzzy searching

Using fuzzystrmatch or fuzzy searching

```
SELECT levenshtein('GUMBO', 'GAMBOL');
```

```
+-------------+
| levenshtein |
|-------------|
|           2 |
+-------------+
```

The levenshtein function calculates the levenshtein distance between two strings which is the number of edits required for the strings to be a perfect match.

Compare two strings with pg_trgm

```
SELECT similarity('GUMBO', 'GAMBOL');
```

```
+------------+
| similarity |
|------------|
| 0.18181818 |
+------------+
```

The pg_trgm extension provides functions and operators to determine the similarity of two strings using trigram matchings. Trigrams are groups of 3 consecutive characters in a string and based on the number of matching trigrams in two strings will provide a measurement of how similar they are. Value is between 0-1, 0 is no matching trigrams, 1 is perfect match